《最优化算法与理论》实验报告

实验序号: 3

实验内容B 步长、拟牛顿法与共轭梯度法

班　　级: 信息 2301

学　　号: 202310040128

姓　　名: 雷龙浩

完成日期: 2026 年 03 月 31 日

目录

1 实验任务与实现说明 **3**
1.1 算法原理简述 3
1.2 程序结构与关键代码 4

2 测试问题与参数设置 **10**
2.1 测试函数 10
2.2 算法参数 10
2.3 实验执行步骤 10

3 实验结果 **11**
3.1 BB 步长最速下降法（Rosenbrock） 11
3.2 Armijo + BFGC/FR（Powell 奇异函数） 11

1 实验任务与实现说明

本次上机练习的两项核心任务如下：

- 实现采用 BB 步长的最速下降法，求解 Rosenbrock 函数极小化问题；
- 采用 Armijo 回溯步长搜索，实现 BFGS（代码中函数名为 `bfgc`）与 FR 共轭梯度法（`fr`），求解 Powell 奇异函数。

实验代码入口为 `work3/work3.py`，运行命令为：

```
python work3/work3.py
```

脚本会自动输出结果表，并在 `work3/picture/` 下生成收敛曲线与轨迹图。

1.1 算法原理简述

1) BB 步长思想

对于最速下降法迭代

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k),$$

BB 方法通过两点信息近似曲率，定义

$$s_k = x_k - x_{k-1}, \qquad y_k = \nabla f(x_k) - \nabla f(x_{k-1}),$$

并用

$$\alpha_k^{\mathrm{BB1}} = \frac{s_k^T s_k}{s_k^T y_k}, \qquad \alpha_k^{\mathrm{BB2}} = \frac{s_k^T y_k}{y_k^T y_k}$$

给出步长候选。本实验代码中默认采用 `auto` 策略，在正值候选里取更保守值。

2) Armijo 回溯条件

$$f(x_k + \alpha d_k) \le f(x_k) + \sigma_1 \alpha \nabla f(x_k)^T d_k, \quad 0 < \sigma_1 < 0.5.$$

若条件不满足，则执行 $\alpha \leftarrow \rho\alpha$（$0 < \rho < 1$）回溯缩步。

3) BFGS 与 FR

$$B_k d_k = -g_k, \qquad B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}.$$

FR 共轭梯度法使用

$$\beta_k^{\mathrm{FR}} = \frac{g_{k+1}^T g_{k+1}}{g_k^T g_k}, \qquad d_{k+1} = -g_{k+1} + \beta_k^{\mathrm{FR}} d_k.$$

1.2 程序结构与关键代码

本次实验采用“目标函数定义 + 优化器 + 步长搜索 + 实验脚本”分层结构。核心文件如下：

- `work3/work3.py`：组织实验、绘图、输出表格；
- `optimization/line_search/bb.py`：BB 步长实现；
- `optimization/optimizers/BFGC.py`：BFGS 优化器实现；
- `optimization/optimizers/FR.py`：FR 共轭梯度实现。

代码展示 1：work3 主实验流程（两组实验）

Listing 1: work3.py 中实验主流程与参数配置

```python
def main() -> None:
    picture_dir = _picture_dir()

    # 实验 1: BB 步长最速下降法求解 Rosenbrock。
    rosen = rosenbrock_objective()
    x0_rosen = np.array([-1.2, 1.0], dtype=float)
    rosen_result, rosen_x_trace, rosen_f_trace, _ = run_with_trace(
        optimizer=steepest_descent,
        x0=x0_rosen,
        objective=rosen,
        line_search=bb_step_search,
        optimizer_params={"max_outer_iter": 20_000, "grad_tol": 1e-6},
        line_search_params={"alpha_init": 0.1, "variant": "auto"},
    )

    _plot_rosenbrock_bb_contour(rosen_x_trace, picture_dir / "work3_rosenbrock_bb_contour.png")
    _plot_convergence_curves(
        {"Steepest + BB": rosen_f_trace},
        "Rosenbrock convergence with BB step size",
        picture_dir / "work3_rosenbrock_bb_curve.png",
    )

    # 实验 2: Armijo 线搜索下，比较 BFGC 与 FR 在 Powell 奇异函数上的表现。
    powell = powell_singular_objective()
    x0_powell = np.array([3.0, -1.0, 0.0, 1.0], dtype=float)
    optimizer_params = {"max_outer_iter": 20_000, "grad_tol": 1e-6}
```

```python
    armijo_params = {"alpha": 1.0, "sigma1": 1e-4, "rho": 0.5}

    powell_bfgc_result, _, powell_bfgc_f_trace, _ = run_with_trace(
        optimizer=bfgc,
        x0=x0_powell,
        objective=powell,
        line_search=armijo_line_search,
        optimizer_params=optimizer_params,
        line_search_params=armijo_params,
    )
    powell_fr_result, _, powell_fr_f_trace, _ = run_with_trace(
        optimizer=fr,
        x0=x0_powell,
        objective=powell,
        line_search=armijo_line_search,
        optimizer_params=optimizer_params,
        line_search_params=armijo_params,
    )

    _plot_convergence_curves(
        {
            "BFGC + Armijo": powell_bfgc_f_trace,
            "FR + Armijo": powell_fr_f_trace,
        },
        "Powell singular function convergence",
        picture_dir / "work3_powell_armijo_curve.png",
    )

    _write_summary(
        _PROJECT_ROOT / "work3" / "work3_summary.csv",
        rosen_result,
        powell_bfgc_result,
        powell_fr_result,
    )

    print("Experiment 1: Steepest + BB on Rosenbrock")
    print(
        f"  iters={rosen_result[2]}, f*={rosen_result[1]:.4e}, "
        f"converged={rosen_result[3]}, grad_norm={rosen_result[4]:.3e}"
    )
```

```python
    print("Experiment 2: Armijo + {BFGC, FR} on Powell singular function")
    print(
        f"  BFGC: iters={powell_bfgc_result[2]}, f*={powell_bfgc_result[1]:.4e}, "
        f"converged={powell_bfgc_result[3]}, grad_norm={powell_bfgc_result[4]:.3e}"
    )
    print(
        f"  FR: iters={powell_fr_result[2]}, f*={powell_fr_result[1]:.4e}, "
        f"converged={powell_fr_result[3]}, grad_norm={powell_fr_result[4]:.3e}"
    )


if __name__ == "__main__":
    main()
```

代码展示 2：BB 步长搜索实现

Listing 2: BB 步长计算核心代码

```python
def bb_step_search(
    xk: np.ndarray,
    _dk: np.ndarray,
    objective: Objective,
    x_prev: np.ndarray | None = None,
    alpha_init: float = 0.1,
    variant: str = "auto",
    eps: float = 1e-12,
    **_: object,
) -> Tuple[float, int]:
    if x_prev is None:
        return float(alpha_init), 0

    xk = np.asarray(xk, dtype=float)
    x_prev = np.asarray(x_prev, dtype=float)
    sk = xk - x_prev
    yk = objective.gradient(xk) - objective.gradient(x_prev)

    sTy = float(np.dot(sk, yk))
    sTs = float(np.dot(sk, sk))
```

```
    yTy = float(np.dot(yk, yk))

    alpha_bb1 = None
    alpha_bb2 = None
    if abs(sTy) > eps:
        alpha_bb1 = sTs / sTy
    if yTy > eps:
        alpha_bb2 = sTy / yTy

    variant_key = variant.strip().lower()
    if variant_key == "bb1":
        alpha = alpha_bb1
    elif variant_key == "bb2":
        alpha = alpha_bb2
    elif variant_key == "auto":
        candidates = [a for a in (alpha_bb1, alpha_bb2) if a is not
            None and a > 0.0]
        alpha = min(candidates) if candidates else None
    else:
        raise ValueError("variant must be one of {'bb1', 'bb2', 'auto
            '}.")

    if alpha is None or (not np.isfinite(alpha)) or alpha <= 0.0:
        alpha = alpha_init

    return float(alpha), 0
```

代码展示 3：BFGC 与 FR 算法核心迭代

Listing 3: BFGC.py 中 BFGS 主迭代代码

```
def bfgc(
    x0: np.ndarray,
    objective: Objective,
    line_search_func: LineSearch = wolfe_powell_line_search,
    grad_tol: float = 1e-6,
    max_outer_iter: int = 1000,
    callback: Optional[IterationCallback] = None,
    **ls_params,
) -> Tuple[np.ndarray, float, int, bool, float]:
    xk = np.asarray(x0, dtype=float)
    n = xk.size
    bk = np.eye(n, dtype=float)
```

```
    gk = objective.gradient(xk)
    converged = False

    for iteration in range(max_outer_iter):
        grad_norm = float(np.linalg.norm(gk))
        if grad_norm < grad_tol:
            converged = True
            break

        dk = _solve_bfgs_direction(bk, gk)
        if float(np.dot(gk, dk)) >= 0.0:
            dk = -gk

        alpha, _ = line_search_func(xk, dk, objective, **ls_params)
        x_next = xk + alpha * dk
        g_next = objective.gradient(x_next)

        sk = x_next - xk
        yk = g_next - gk
        bk = _bfgs_update(bk, sk, yk)

        if callback is not None:
            callback(iteration + 1, x_next.copy(), objective.value(
                x_next), grad_norm, float(alpha))

        xk = x_next
        gk = g_next
    else:
        iteration = max_outer_iter
        grad_norm = float(np.linalg.norm(gk))

    return (xk, objective.value(xk), iteration, converged, grad_norm)
```

Listing 4: FR.py 中 Fletcher-Reeves 主迭代代码

```
def fr(
    x0: np.ndarray,
    objective: Objective,
    line_search_func: LineSearch = wolfe_powell_line_search,
    grad_tol: float = 1e-6,
    max_outer_iter: int = 1000,
    callback: Optional[IterationCallback] = None,
```

```
    **ls_params,
) -> Tuple[np.ndarray, float, int, bool, float]:
    xk = np.asarray(x0, dtype=float)
    gk = objective.gradient(xk)
    dk = -gk
    converged = False
    eps = 1e-16

    for iteration in range(max_outer_iter):
        grad_norm = float(np.linalg.norm(gk))
        if grad_norm < grad_tol:
            converged = True
            break

        if float(np.dot(gk, dk)) >= 0.0:
            dk = -gk

        alpha, _ = line_search_func(xk, dk, objective, **ls_params)
        x_next = xk + alpha * dk
        g_next = objective.gradient(x_next)

        denom = float(np.dot(gk, gk))
        if denom <= eps:
            beta = 0.0
        else:
            beta = float(np.dot(g_next, g_next) / denom)
            beta = max(beta, 0.0)

        d_next = -g_next + beta * dk
        if float(np.dot(g_next, d_next)) >= 0.0:
            d_next = -g_next

        if callback is not None:
            callback(iteration + 1, x_next.copy(), objective.value(
                x_next), grad_norm, float(alpha))

        xk = x_next
        gk = g_next
        dk = d_next
    else:
        iteration = max_outer_iter
```

```
        grad_norm = float(np.linalg.norm(gk))

    return (xk, objective.value(xk), iteration, converged, grad_norm)
```

2 测试问题与参数设置

2.1 测试函数

1) Rosenbrock 函数

$$f(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2, \quad x_0 = (-1.2, 1.0)^T. \tag{1}$$

2) Powell 奇异函数

$$f(x) = (x_1 + 10x_2)^2 + 5(x_3 - x_4)^2 + (x_2 - 2x_3)^4 + 10(x_1 - x_4)^4, \tag{2}$$

$$x_0 = (3, -1, 0, 1)^T, \quad x^* = (0, 0, 0, 0)^T. \tag{3}$$

2.2 算法参数

表 1: 实验参数配置

实验	方法	参数	取值
Rosenbrock	Steepest + BB	ϵ, max_iter	10^{-6}, 20000
Rosenbrock	BB 步长参数	alpha_init, variant	0.1, auto
Powell	BFGC + Armijo	ϵ, max_iter	10^{-6}, 20000
Powell	FR + Armijo	ϵ, max_iter	10^{-6}, 20000
Powell	Armijo 参数	α_0, σ_1, ρ	1.0, 10^{-4}, 0.5

2.3 实验执行步骤

为保证结果可复现，实验流程统一为：

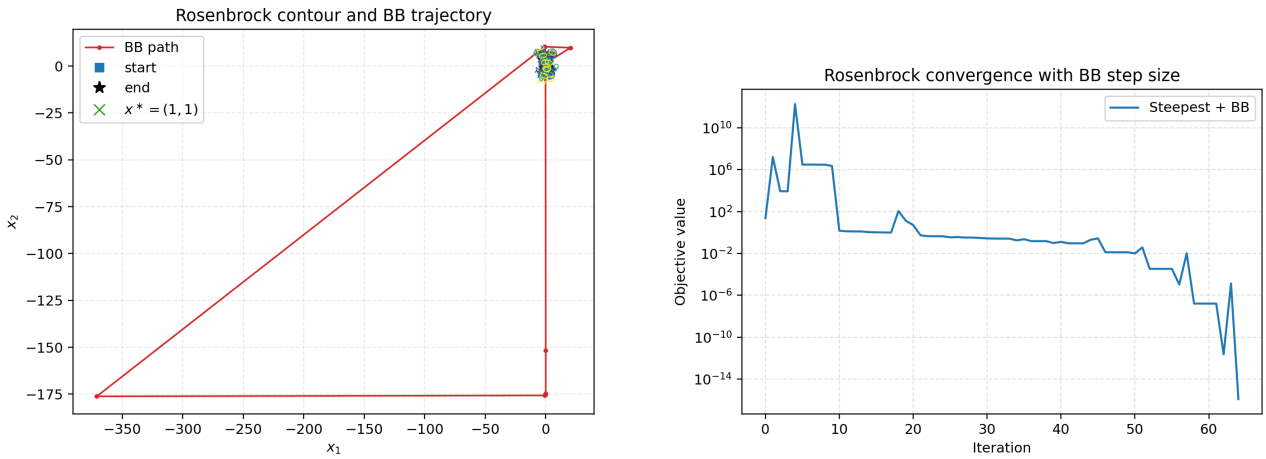
1. 在项目根目录运行 `python work3/work3.py`；
2. 自动生成三张图片：`work3_rosenbrock_bb_contour.png`、`work3_rosenbrock_bb_curve.png`、`work3_powell_armijo_curve.png`；
3. 自动生成 `work3/work3_summary.csv`，其中包含迭代次数、最终函数值与梯度范数。

3 实验结果

3.1 BB 步长最速下降法（Rosenbrock）

表 2: Steepest + BB 在 Rosenbrock 上的结果

方法	迭代次数	f^*	是否收敛	$\|\nabla f(x^*)\|$
Steepest + BB	64	1.2266×10^{-16}	是	1.6053×10^{-8}



(a) Rosenbrock 等高线与 BB 迭代轨迹

(b) Rosenbrock 目标函数值收敛曲线

图 1: BB 步长在 Rosenbrock 问题上的优化过程。

从图 1 可以看到，BB 步长在谷底附近仍存在一定摆动，但相比固定步长最速下降法，收敛速度明显提升，能够在较少迭代内达到较高精度。

3.2 Armijo + BFGC/FR（Powell 奇异函数）

表 3: Powell 奇异函数上的算法对比

方法	迭代次数	f^*	是否收敛	$\|\nabla f(x^*)\|$
BFGC + Armijo	29	3.2600×10^{-10}	是	7.3476×10^{-7}
FR + Armijo	115	4.7300×10^{-12}	是	6.5085×10^{-7}

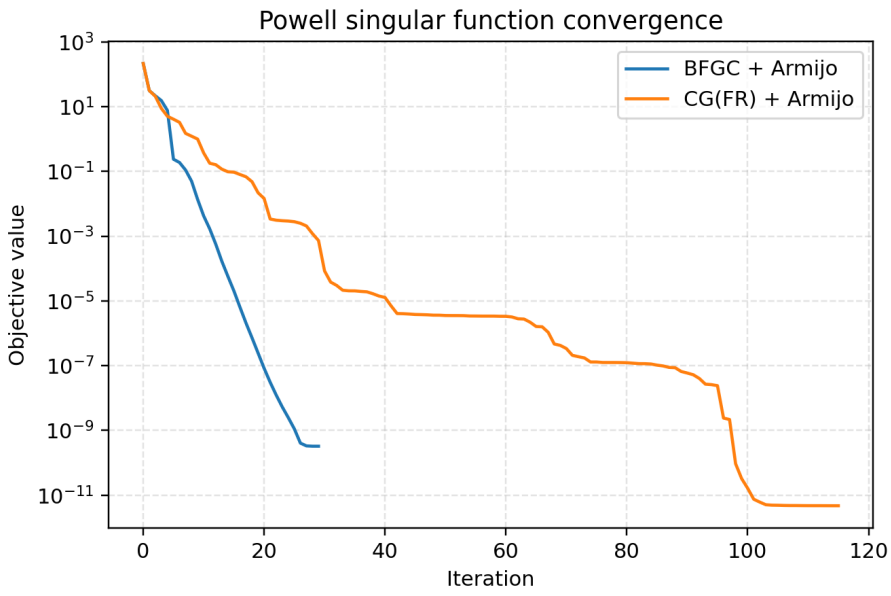


图 2: Powell 奇异函数上，BFGC 与 FR 在 Armijo 步长搜索下的收敛曲线。

由表 3 和图 2 可见：BFGC 迭代次数更少，表现出拟牛顿法利用曲率信息的优势；FR 迭代次数较多，但最终函数值更低，说明在该问题上其后期下降能力也较强。